

Code-Layout, Readability and Reusability

Date	11 July 2026
Team ID	
Project Name	EduGenie – Google Gemini Powered Learning Assistant
Maximum Marks	5 Marks

Code Layout Checklist

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	PEP 8 style followed with consistent 4-space indentation.
2	Proper File Structure	Files and folders are logically organized	Yes	Organized into app/, routes/, services/, templates/, and static/ folders.
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	Descriptive names used throughout (e.g., quiz_topic, summary_text).
4	Function / Method Names	Functions are descriptively named	Yes	Names clearly indicate purpose (e.g., generate_quiz(), summarize_text()).
5	Code Comments	Inline and block comments explain logic	Yes	Docstrings and inline comments added for key functions and AI prompt logic.
6	Modular Design	Code is split into reusable functions/modules	Yes	Separate modules for Q&A, explanation, quiz, summarization, and learning path.
7	No Redundant Code	Duplicate or unused code is removed	Yes	Common logic refactored into shared helper/utility functions.
8	Error Handling	Exceptions and errors are handled gracefully	Yes	Try/except blocks around API calls and input validation with user-friendly error messages.

Reusable Components / Modules

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High / Medium / Low)
1	Gemini API Client Wrapper	Python (FastAPI)	Used across Q&A, Concept Explanation, Quiz Generation, Summarization, and Learning Path modules	High
2	Prompt Template Utility	Python	Used to build structured prompts for all Generative AI features	High
3	Response Formatter	Python	Formats AI responses into JSON for the frontend	Medium
4	Quiz Generator Module	Python	Reused for topic-specific and difficulty-based quiz creation	Medium
5	UI Components (Chat Bubble, Card)	HTML / CSS	Reused across the question-answering, quiz, and summarization interfaces	High

Overall Code Quality Assessment

Aspect	Rating (1–5)	Comments
Code Layout & Structure	4	Well-organized, modular FastAPI project structure with clear separation of concerns.
Readability	4	Clear, descriptive naming and consistent formatting throughout the codebase.
Reusability	4	Common utilities (API client, prompt templates) reused across multiple AI features.
Documentation / Comments	3	Core functions documented; additional inline comments recommended for complex logic.
Overall Score	4	Solid, maintainable codebase with room for expanded documentation.